

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Problem Solving Assignment

COURSE NAME: **ARTIFICIAL INTELLIGENCE COURSE CODE:** **MLA01**
 AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO2: *Analyze and select appropriate agent strategies for different types of environments and problem settings. (BTL 3)*

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 60 Mins

No. of Questions: 5

Annexure A
Problem Solving Assignment

Code of Conduct:

I (V Sunil Kumar / 192425292) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V Sunil Kumar

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

QUESTION 1 – SMART AMBULANCE NAVIGATION

1. Problem Analysis and Understanding

An ambulance must travel from a hospital to an emergency location through a city road network. Roads have different travel costs and an estimated distance to the destination is available for each junction. The objective is to find the lowest-cost route from the hospital to the emergency location.

2. AI Algorithm Selection & Justification

A* (A-star) informed search is appropriate because both the cost already travelled and an estimate of the remaining distance are available. A* evaluates a junction using:

$$f(n) = g(n) + h(n)$$

Here, $g(n)$ is the actual cost from the hospital to junction n , $h(n)$ is the heuristic estimate from n to the emergency location, and $f(n)$ is the estimated total route cost. If $h(n)$ is admissible, A* can return an optimal route.

3. Algorithm Design / Pseudocode

A_STAR(start, goal):

OPEN = {start}

CLOSED = {}

$g(\text{start}) = 0$

$f(\text{start}) = g(\text{start}) + h(\text{start})$

while OPEN is not empty:

n = node in OPEN with minimum $f(n)$

if n == goal:

return path from start to goal

remove n from OPEN

add n to CLOSED

for each neighbor m of n:

new_g = $g(n) + \text{cost}(n,m)$

if m is new or new_g < $g(m)$:

parent(m) = n

$g(m) = \text{new_g}$

$f(m) = g(m) + h(m)$

add m to OPEN

return failure

4. Application of A*

Start with the hospital junction.

Calculate g, h and f for every reachable neighboring junction.

Choose the junction with the smallest f-value.

Expand that junction and update the costs of its neighbors.

Repeat until the emergency location is selected.

Trace parent links to obtain the selected route.

5. State-Space Representation

Hospital $\rightarrow$ Junction A $\rightarrow$ Junction B $\rightarrow$ Junction C $\rightarrow$ Emergency

$\searrow$ Junction D $\rightarrow$ Junction E $\nearrow$

The actual junction names, road costs, heuristic values, explored sequence and numerical f-values must be taken from the road network supplied with the question. The uploaded PDF text does not contain that network, so exact numerical values cannot be stated without inventing data.

6. Complexity Analysis and Efficiency

A* can be efficient when the heuristic guides the search toward the goal. With an appropriate admissible and consistent heuristic, it avoids unnecessary exploration compared with uninformed search. The exact complexity depends on the graph representation and heuristic; in general, the number of expanded nodes is the key efficiency factor.

1. Problem Analysis and Understanding

The robot must move from a charging station to a target shelf. It does not initially know all obstacles and discovers some obstacles only when it reaches nearby cells. Therefore, the environment is partially unknown during planning.

2. AI Algorithm Selection & Justification

An Online Search Agent is suitable because the robot must act while learning about the environment. It can use informed search ideas to select promising moves, update its internal map after sensing an obstacle, and re-plan.

3. Algorithm Design / Pseudocode

ONLINE_SEARCH(start, goal):

current = start

known_map = initial_information

while current != goal:

observe nearby cells

update known_map

if planned next cell is blocked:

revise the path

choose the best reachable next cell

move to that cell

current = next cell

return success

4. Step-by-Step Application

Begin at the charging station.

Use the currently known grid to select a promising path.

Move to the next cell and observe the surroundings.

If an obstacle is discovered, mark that cell as blocked.

Discard the affected part of the old plan and re-plan.

Continue until the target shelf is reached.

5. State-Space Representation

Charging Station → Cell 1 → Cell 2 → ?

↓

Obstacle discovered

↓

Update map and re-plan

↓

Cell 3 → Target Shelf

6. Online Search vs Offline Search

Online Search	Offline Search
Environment initially unknown	Environment is known before planning
Learns while acting	Plans using the complete model
Reacts to newly discovered obstacles	Normally does not need discovery-based re-planning
Suitable for dynamic/unknown situations	Suitable for known environments

7. Complexity and Efficiency

Online search may perform extra work because it can need re-planning after new information is discovered. Its major advantage is adaptability: it can continue operating even when the initial map is incomplete.

1. Problem Analysis and Understanding

Six examinations E1–E6 must be assigned to four time slots T1–T4. Examinations sharing students cannot occur in the same slot. Hall capacities must also be respected.

2. CSP Formulation

Variables: E1, E2, E3, E4, E5, E6

Domain of every variable: {T1, T2, T3, T4}

Objective: assign every examination to a time slot without violating student-conflict and hall-capacity constraints.

3. Constraint Identification & Representation

$E1 \neq E2$

$E1 \neq E3$

$E2 \neq E4$

$E3 \neq E5$

$E4 \neq E6$

$E2 \neq E5$

Hall capacities: Hall A = 120, Hall B = 90, Hall C = 60 students.

Exam sizes: E1 = 100, E2 = 80, E3 = 70, E4 = 60, E5 = 90, E6 = 50 students.

4. Backtracking + MRV + Forward Checking

Initial domains:

$E1 = \{T1, T2, T3, T4\}$

$E2 = \{T1, T2, T3, T4\}$

$E3 = \{T1, T2, T3, T4\}$

$E4 = \{T1, T2, T3, T4\}$

$E5 = \{T1, T2, T3, T4\}$

$E6 = \{T1, T2, T3, T4\}$

All variables initially have four values, so select E1. After each assignment, Forward Checking removes the selected slot from the domains of examinations directly constrained by that assignment.

Step	Assignment	Forward-Checking Domain Update
1	$E1 = T1$	$E2 = \{T2, T3, T4\};$ $E3 = \{T2, T3, T4\}$
2	$E2 = T2$	$E4 = \{T1, T3, T4\};$ $E5 = \{T1, T3, T4\}$
3	$E3 = T2$	$E5 = \{T1, T3, T4\}$

4	$E4 = T1$	$E6 = \{T2, T3, T4\}$
5	$E5 = T1$	No conflict with E1 because E1 and E5 are not constrained to differ
6	$E6 = T2$	No conflict with E4 because $E4 = T1$

5. Final Timetable

Time Slot	Examinations	Hall Assignment
T1	E1, E4, E5	$E1 \rightarrow \text{Hall A}; E4 \rightarrow \text{Hall C}; E5 \rightarrow \text{Hall B}$
T2	E2, E3, E6	$E2 \rightarrow \text{Hall B}; E3 \rightarrow \text{Hall A}; E6 \rightarrow \text{Hall C}$
T3	—	—
T4	—	—

The timetable is conflict-free under the six specified student-conflict constraints. Each examination is also assigned to a hall that can accommodate its student count.

6. Complexity Analysis

Basic backtracking has worst-case exponential behavior, approximately $O(d^n)$, where n is the number of examinations and d is the number of available time slots. MRV reduces the branching choices by selecting the most constrained variable, while Forward Checking detects domain failures early. These techniques therefore improve practical efficiency.

1. Problem Analysis and Understanding

The AI must select the best move from a game tree. Terminal states contain scores. The opponent is assumed to play optimally, so the decision should be obtained using Minimax.

2. Algorithm Selection & Justification

Minimax is appropriate for deterministic, two-player, adversarial games. Alpha-Beta Pruning can then reduce the number of nodes that must be evaluated without changing the final Minimax decision.

3. Minimax Pseudocode

MINIMAX(node, maximizing):

if node is terminal:

return utility(node)

if maximizing:

value = $-\infty$

for each child:

value = max(value, MINIMAX(child, false))

return value

else:

value = $+\infty$

for each child:

value = min(value, MINIMAX(child, true))

return value

4. Alpha-Beta Pseudocode

ALPHA_BETA(node, alpha, beta, maximizing):

if node is terminal:

return utility(node)

if maximizing:

for child:

alpha = max(alpha, ALPHA_BETA(child, alpha, beta, false))

if alpha $\geq$ beta:

break

return alpha

else:

for child:

beta = min(beta, ALPHA_BETA(child, alpha, beta, true))

if beta $\leq$ alpha:

```
break
return beta
```

5. Search-Tree Representation

```
MAX
/\
MIN MIN
/|\|/\
leaf ... leaf ...
```

The exact terminal scores, selected move and branches pruned by Alpha-Beta must be taken from the supplied game tree. That game tree is not present in the uploaded PDF text, so exact numerical results cannot be supplied without inventing the missing tree.

6. Complexity Analysis

For a game tree with branching factor b and depth m , ordinary Minimax has time complexity $O(b^m)$ and space complexity $O(bm)$ for depth-first implementation. Alpha-Beta has worst-case time $O(b^m)$, but with excellent move ordering its effective search can approach $O(b^{(m/2)})$, allowing deeper search with the same computational resources.

1. Problem Analysis and Understanding

A delivery vehicle must visit multiple customers while minimizing total travel distance. The initial route is improved by examining neighboring routes.

2. Algorithm Selection & Justification

Hill Climbing is a local-search optimization method. It is suitable when a route can be represented as a state and nearby routes can be generated by small changes in customer order.

3. Algorithm Design / Pseudocode

```
HILL_CLIMBING(initial_route):
```

```
    current = initial_route
```

```
    while true:
```

```
        neighbors = generate_neighbor_routes(current)
```

```
        best = neighbor with minimum distance
```

```
    if distance(best) >= distance(current):
```

```
        return current
```

```
    current = best
```

4. Step-by-Step Application

Route	Distance	Decision
A→B→C→D→A	100 km	Initial route
A→B→D→C→A	90 km	Improvement
A→C→B→D→A	85 km	Best neighbor; select
A→D→C→B→A	95 km	Not selected

After selecting A→C→B→D→A at 85 km, if every neighboring route has distance 85 km or greater, Hill Climbing stops.

5. State-Space / Neighborhood Representation

Initial Route (100)

↓

Neighbors: 90, 85, 95

↓

Select 85

↓

Check new neighbors

↓

No better neighbor → Stop

6. Global Optimum and Local Optimum

The final route produced by Hill Climbing is not guaranteed to be a global optimum. The algorithm may stop at a local optimum when every immediate neighbor is worse even though a better route exists elsewhere in the search space.

7. Escaping a Local Optimum

Random Restart: run Hill Climbing from different initial routes and select the best result.

Stochastic Hill Climbing: choose among improving moves probabilistically.

Simulated Annealing: occasionally accepts worse moves to escape local optima.

8. Complexity and Efficiency

The efficiency depends on how many neighboring routes are generated and evaluated at each step. Hill Climbing normally uses much less memory than exhaustive search because it stores the current state and its local neighborhood rather than the complete search tree. However, it is not complete and does not guarantee a global optimum.

Criteria	Weightage (%)	Excellent (Full Marks)	Good	Average	Poor
CSP Problem Formulation	20%	4% – Clearly defines all variables, domains, and problem objectives correctly.	3% – Defines most variables and domains with minor errors.	2% – Partial formulation with some missing components.	0–1% – Incorrect or incomplete formulation.
Constraint Identification & Representation	20%	4% – Accurately identifies all student-conflict and hall-capacity constraints and represents them correctly.	3% – Identifies most constraints with minor errors.	2% – Identifies some constraints but misses important relationships.	0–1% – Constraints are largely incorrect or missing.
Application of Backtracking, MRV & Forward Checking	30%	6% – Correctly applies Backtracking, MRV, and Forward Checking with clear and logical steps.	4.5% – Applies the techniques correctly with minor mistakes.	3% – Partial application with unclear or inconsistent steps.	0–1.5% – Incorrect or inappropriate application of CSP techniques.
Step-by-Step Execution & Domain Updates	20%	4% – Provides a complete, logical execution trace and correctly updates domains after every assignment.	3% – Mostly correct execution with minor gaps.	2% – Provides limited steps with some incorrect domain updates.	0–1% – Incomplete or incorrect execution.
Final Timetable, Accuracy & Justification	10%	2% – Produces a valid conflict-free timetable satisfying all constraints and capacity requirements with clear justification.	1.5% – Valid timetable with minor errors in justification.	1% – Partially valid timetable with some constraint violations.	0–0.5% – Invalid or missing timetable.